Design Rationale for Text-Based Adventure Game

1. Software Design

Our project is a text-based adventure game where the player moves through different environments, solves puzzles, collects items, and fights enemies to win. The game is built using object-oriented programming (OOP) to keep every part organized and reusable.

At the top, the GameManager controls everything — it starts the game (startGame()), moves to new areas (nextStage()), and ends the story (endGame()). It connects with three main systems:
• PuzzleManager – loads puzzles, checks answers, and gives rewards.
• CombatSystem – manages battles between the player and NPCs.
• RewardSystem – gives points and upgrades weapons.

The Environment classes (River, Forest, Cave) each represent a different stage of the story. They all come from the abstract Environment class and share methods like enter() and triggerEvent(), but each has its own unique challenge.

The Player class represents the main character who can pick up and use Items like a Lamp or Sword, stored inside the Inventory. Both Player and NPCs inherit from the abstract Character class, which defines shared attributes such as name, health, and interact().

To keep behaviors separate, three small interfaces define special actions:
• IInteractable – for interacting with NPCs.
• IFightable – for characters that can battle.
• ICollectible – for items the player can collect.

This modular structure keeps the design clear and easy to expand — for example, new puzzles, items, or environments can be added without changing existing code.

2. Design Rationale

Our UML design follows the **SOLID** and **DRY** principles to make the system structured, flexible, and easy to maintain. Each class has a clear purpose, and the relationships between them support good software design practices.

S – Single Responsibility

Every class in the design focuses on one task.

- `GameManager` handles the game flow (start, next stage, end).
- `PuzzleManager` loads and checks puzzles.
- `CombatSystem` controls fights between Player and NPC.

- `RewardSystem` manages points and upgrades.
  This separation keeps the code simple, easy to test, and prevents one change from affecting other parts of the game.

O – Open/Closed

The system can grow without modifying existing code.
New puzzles, tools, or environments can be added by extending existing abstract classes like `Environment`, `Item`, or `Puzzle`. For example, creating a new class `Mountain` from `Environment` adds new gameplay without touching old code.

L – Liskov Substitution

Subclasses behave like their parent classes.
Environments (`River`, `Cave`, `Forest`) all use shared methods like `enter()` and `triggerEvent()`.
Items (`Lamp`, `Key`) inherit from `Item` and can be used in the same way by the player.
This makes the gameplay consistent and predictable.

I – Interface Segregation

Small, focused interfaces are used instead of one large one.

- `IInteractable` is for talking or interacting with NPCs.
- `IFightable` is for characters that can battle.
- `ICollectible` is for items the player can pick up.
  This keeps each class lightweight and avoids forcing it to include unnecessary functions.

D – Dependency Inversion

High-level classes depend on abstract ones, not concrete ones.
`GameManager` interacts with `Environment`, `PuzzleManager`, and `CombatSystem` generally — it doesn't rely on specific subclasses like `Cave` or `Forest`. This makes the game easy to modify or expand without breaking existing logic.

DRY – Don't Repeat Yourself

Common logic is defined once and reused.
Methods like `enter()`, `use()`, and `triggerEvent()` are written in base classes, then inherited by subclasses.
Central managers (`PuzzleManager`, `RewardSystem`) also prevent duplicate logic across different parts of the game.
This keeps the code clean and reduces errors.